

CS 188: Inteligência Artificial

Problemas de Satisfação de Restrições II

Instrutores: Emma Pierson e Peyrin Kao

Universidade da Califórnia, Berkeley



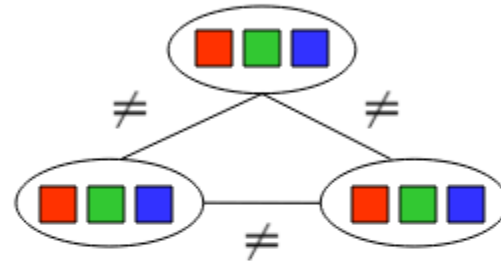
Hoje

- Solução Eficiente de CSPs
- Busca Local

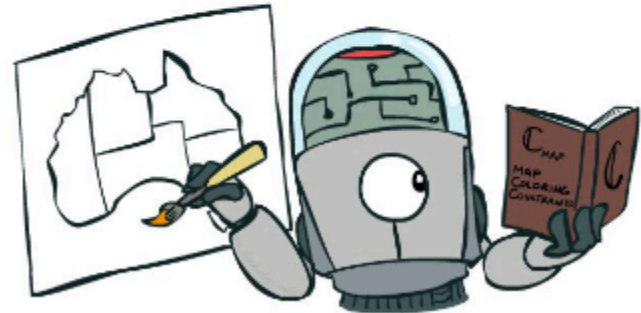


Lembrete: CSPs

- CSPs: ▪ Variáveis ▪ Domínios ▪ Restrições ▪ Implícito (fornecer código para computar) ▪ Explícito (fornecer uma lista das tuplas legais) ▪ Unário / Binário / N-ário



- Objetivos: ▪ Aqui: encontre qualquer solução ▪ Além disso: encontre tudo, encontre o melhor, etc.



Busca de retrocesso

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment given CONSTRAINTS[csp] then
      add {var = value} to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Melhorando o Backtracking

- Ideias de uso geral dão grandes ganhos de velocidade
 - ... mas ainda é tudo NP-difícil



- Filtragem: Podemos detectar falhas

inevitáveis antecipadamente? ▪ Pedido:

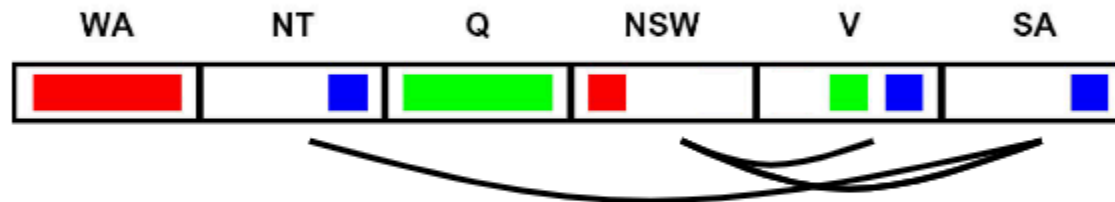
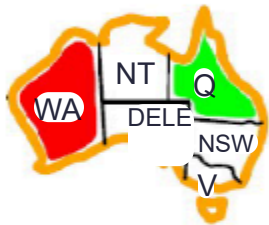
- Qual variável deve ser atribuída em seguida? (MRV) ▪
Em que ordem seus valores devem ser testados? (LCV)
- Estrutura: Podemos explorar a estrutura do problema?



Consistência do Arco e Além

Consistência de arco de um CSP inteiro

- Um arco $X \rightarrow Y$ é consistente se para cada x na cauda existe algum y na cabeça que poderia ser atribuído sem violar uma restrição (cabeça = a seta!)
- Uma forma simples de propagação garante que todos os arcos sejam consistentes:



- Consistência de arco detecta falha antes da verificação
- direta
- Importante: Se X perder um valor, vizinhos de X precisam ser reverificados!
- Deve ser executado novamente após cada tarefa!

*Lembre-se:
Excluir de
A cauda!*

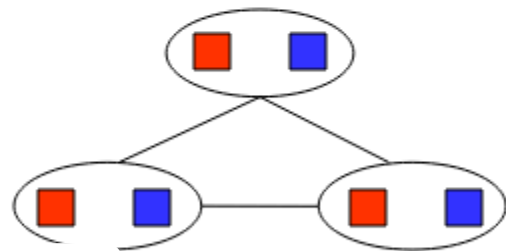
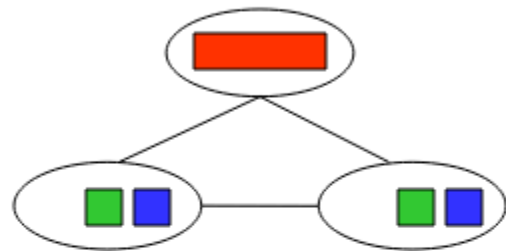
Limitações da Consistência do Arco

- Após impor o arco

consistência:

- Pode sobrar uma solução
- Pode ter várias soluções restantes
- Não pode sobrar nenhuma solução (e não saber)

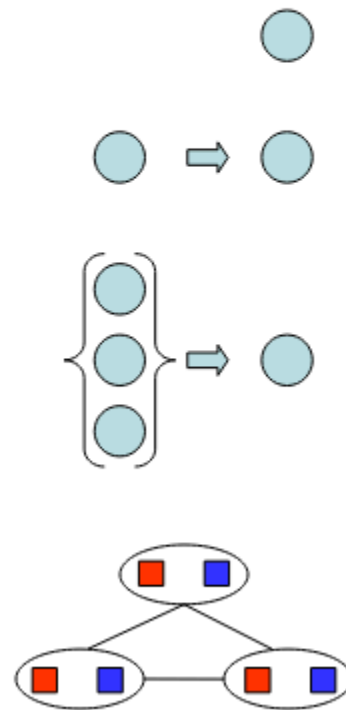
- A consistência de arco ainda é válida
Dentro de uma busca de retrocesso!



O que deu errado aqui?

K-Consistência

- Graus crescentes de consistência
 - 1-Consistência (Consistência de Nó): O domínio de cada nó possui um valor que atende às restrições
 - unárias desse nó
 - 2-Consistência (Consistência de Arco): Para cada par de nós, qualquer atribuição consistente a um pode ser estendida para o outro
 - Como dissemos antes: para qualquer atribuição à cauda, há uma atribuição à cabeça.
 - K-Consistência: Para cada k nós, Qualquer atribuição consistente a K-1 pode ser estendida ao k-ésimo nó.
- K mais alto é mais caro de calcular (mas torna a filtragem melhor: troca!)
- (Você precisa conhecer o caso k=2: consistência do arco)



Forte K - Consistência

▪ Forte k-consistência: também k-1, k-2, ... 1 consistente ▪ Afirmação: forte n-consistência significa que podemos resolver sem voltar atrás! ▪ Por quê? ▪ Escolha qualquer atribuição a qualquer variável ▪ Escolha uma nova variável ▪ Por 2-consistência, há uma escolha consistente com a primeira ▪ Escolha uma nova variável ▪ Por 3-consistência, há uma escolha consistente com as 2 ▪ primeiras ... ▪ Estabelecer uma consistência forte em N é ótimo, mas muito caro ▪ Há muito meio-termo entre consistência em arco e consistência n! (por exemplo, k=3, chamado

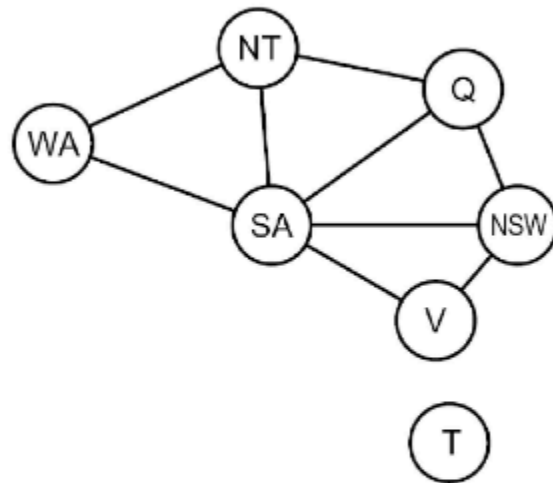
consistência de caminhos)

Estrutura

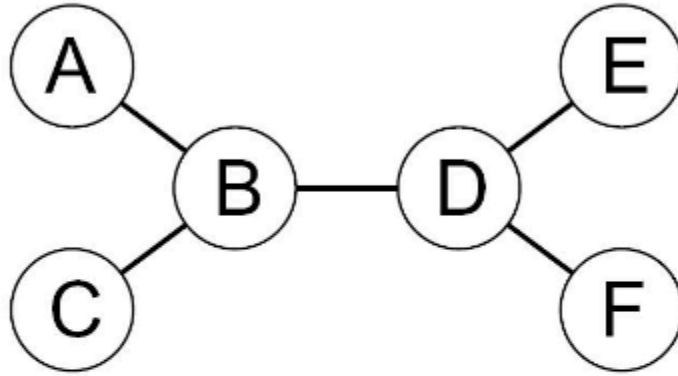


Estrutura do Problema

▪ Caso extremo: subproblemas independentes
Exemplo: Tasmânia e o continente não interagem
Subproblemas independentes são identificáveis como componentes conexos de grafo
de restrição Suponha que um grafo de n variáveis possa ser dividido em subproblemas de apenas c variáveis: ▪ o custo da solução no pior caso é $O((n/c)(dc))$, linear em n ▪ Por exemplo, $n = 80$, $d = 2$, $c = 20$ ▪ $280 = 4$ bilhões de anos a 10 milhões de nós/seg ▪ $(4)(220) = 0,4$ segundos a 10 milhões de nós/seg Raramente, porém, Grafos se decompõem em partes perfeitamente desconexas



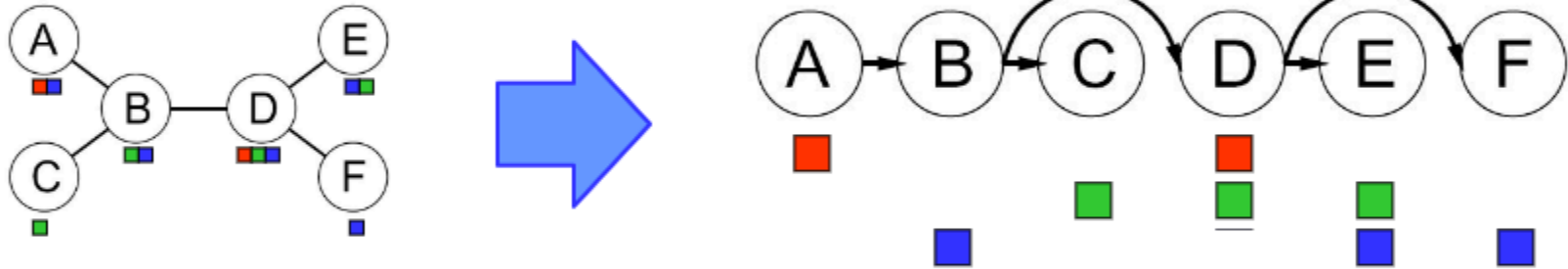
CSPs Estruturados em Árvore



- Teorema: se o grafo de restrições não possui laços, o CSP pode ser resolvido em tempo $O(n \cdot d^2)$. Compare com CSPs gerais, onde o tempo no pior caso é $O(d^n)$.
- Essa propriedade também se aplica ao raciocínio probabilístico (posteriormente): um exemplo da relação entre restrições sintáticas e a complexidade do raciocínio

CSPs Estruturados em Árvore

- Algoritmo para CSPs estruturados em árvore: ▪ Ordem: Escolha uma variável raiz, variáveis de ordem para que os pais precedam os filhos



- Remover para trás: Para $i = n : 2$, aplicar $\text{RemoveInconsistent}(\text{Parent}(X_i), X_i)$
- Atribuir ▪ para frente: Para $i = 1 : n$, atribuir X_i consistentemente com $\text{Parent}(X_i)$
- Duração: $O(n \cdot d^2)$ (por quê?)

CSPs Estruturados em Árvore

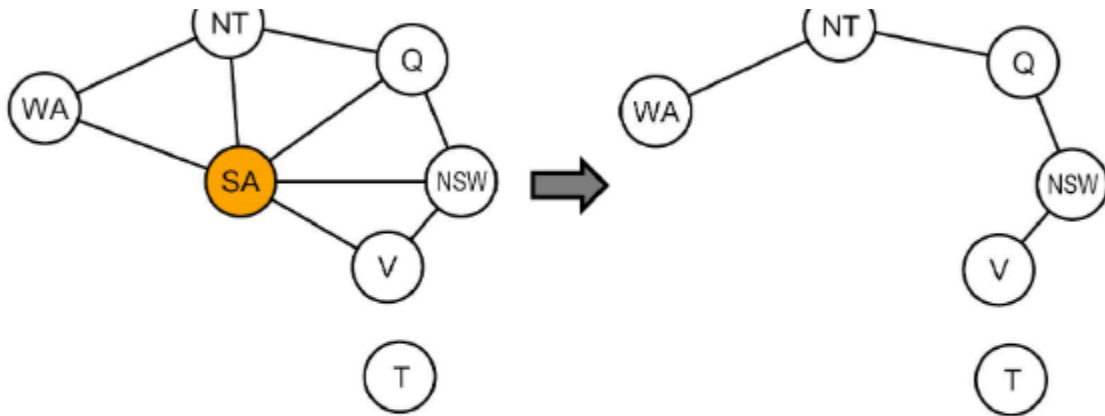
- Afirmação 1: Após a passagem retroativa, todos os arcos da raiz à folha são consistentes ▪ Prova: Cada $X \rightarrow Y$ foi tornado consistente em um ponto e o domínio de Y não poderia ter sido reduzido depois disso (porque os filhos de Y foram processados antes de Y) ▪ Por exemplo: uma vez que $B \rightarrow D$ for consistente, não apagaremos mais nada do domínio de D (o que poderia causar problemas) porque estamos processando na direção oposta



- Afirmação 2: Se os arcos da raiz à folha forem consistentes, a atribuição direta não ▪ recuará Prova: Indução na posição ▪ que quer que atribuíssemos a A , sabemos que há uma atribuição a B ; seja lá o que atribuíssemos a $A+B$, sabemos que há uma atribuição a C ▪. Por que esse algoritmo não funciona com ciclos no grafo de restrições? ▪ Se dois arcos apontam para o mesmo nó, é garantido que ambos tenham atribuições individualmente consistentes – mas essas atribuições podem entrar em conflito. (Dois pais podem discutir!) ▪ Nota: veremos essa ideia básica novamente com as redes de Bayes

Melhorando a Estrutura

CSPs quase estruturados em árvore



- Condicionamento: instanciar uma variável, podar domínios de seus vizinhos
- Condicionamento de corte: instanciar (em todos os aspectos) um conjunto de variáveis de modo que o grafo de restrição restante seja uma árvore
- O tamanho do corte c dá tempo de execução $O((dc)(n-c)d^2)$, muito rápido para c pequeno

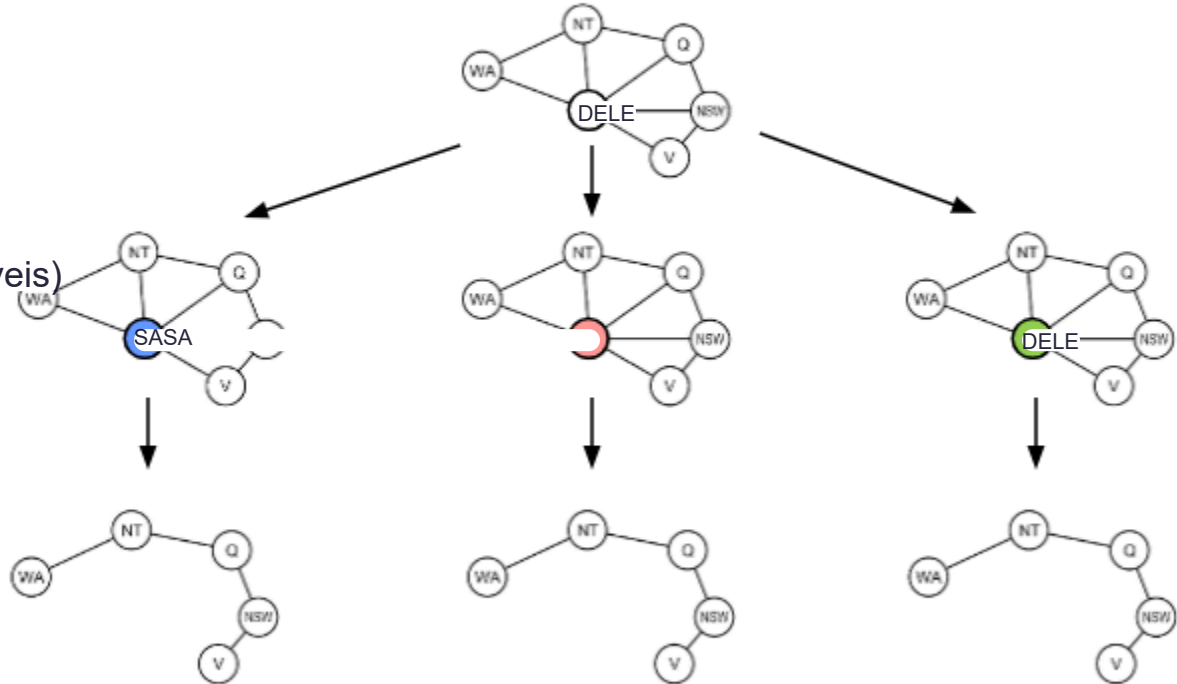
Condicionamento Cutset

Escolha um cutset

Instanciar o cutset
(de todas as formas possíveis)

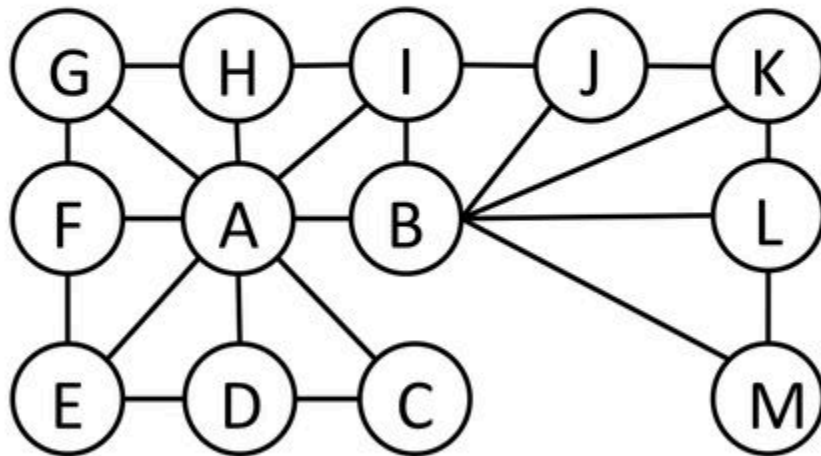
Calcular o CSP residual
para cada tarefa

Resolver os CSPs residuais
(estrutura em árvore)



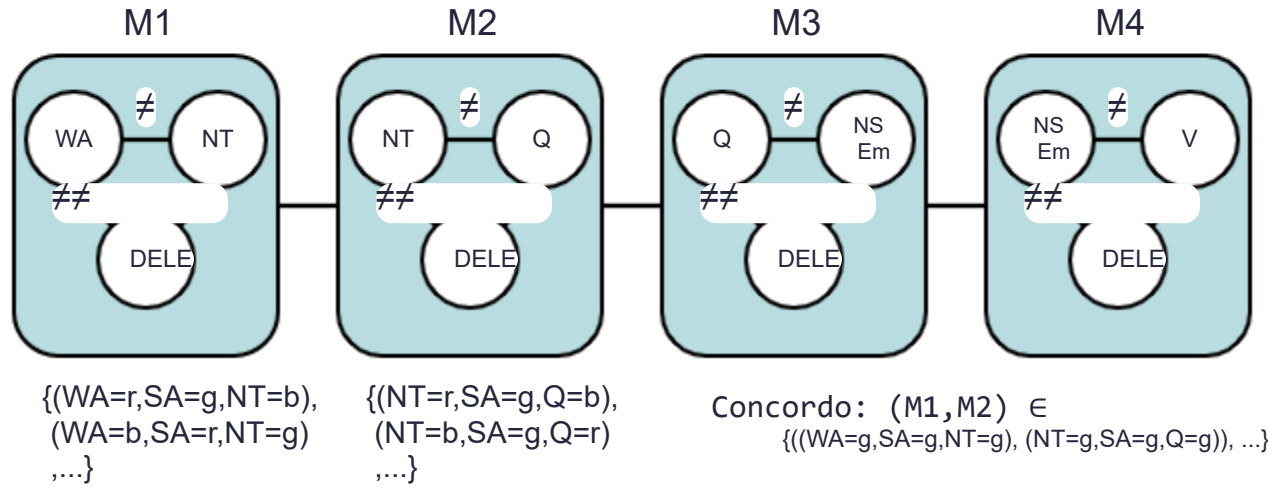
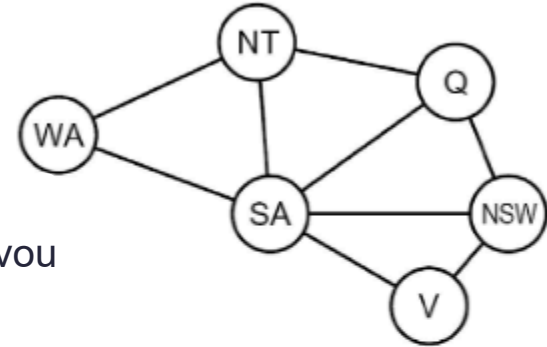
Cutset Quiz

- Encontre o menor cutset que te dê uma árvore para o gráfico abaixo.

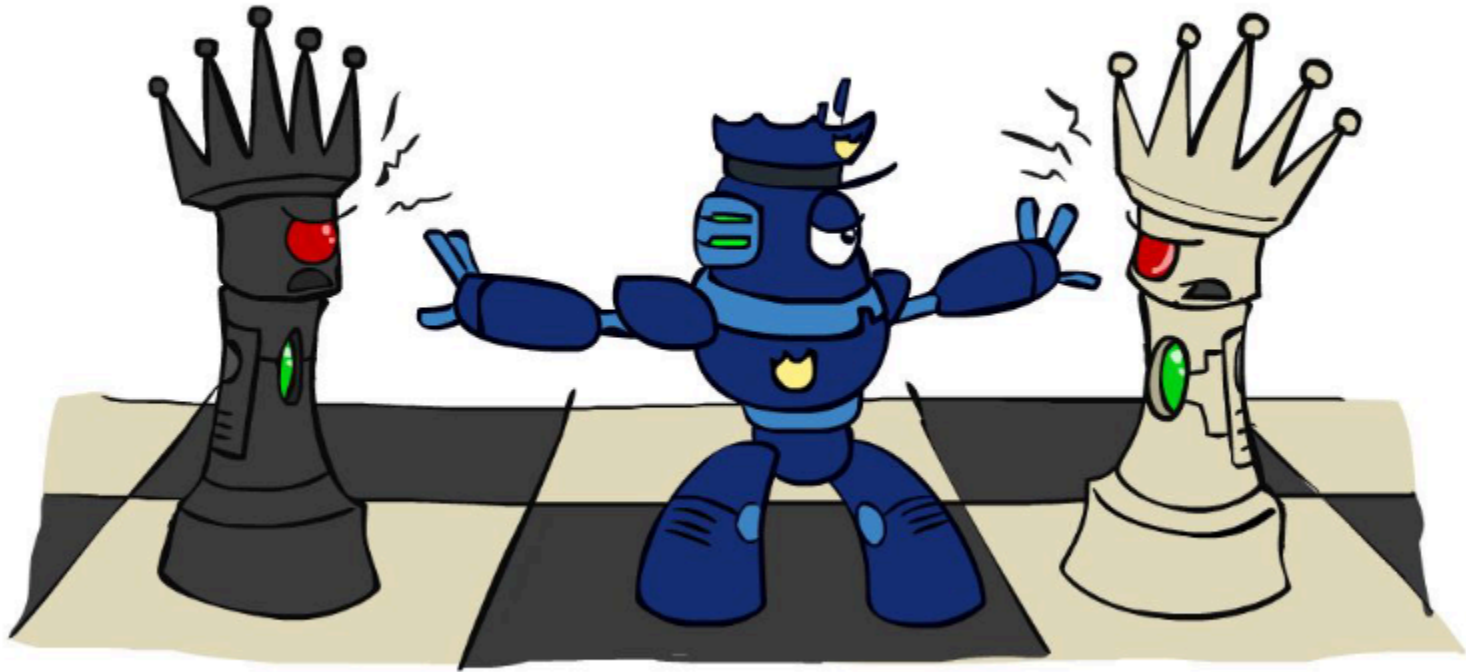


Decomposição em árvore*

- Ideia: criar um grafo estruturado em árvore de megavariáveis▪ Cada megavariável codifica parte da sobreposição original dos subproblemas do CSP▪ para garantir soluções consistentes
- (não dá para resolvê-los separadamente) ▪ É preciso configurar os subproblemas corretamente para garantir que sejam uma árvore (não vou Discuta como fazer isso)



Melhoria Iterativa



Algoritmos Iterativos para CSPs

▪ Métodos de busca local normalmente funcionam com estados "completos", ou seja, todas as variáveis atribuídas

▪ Para aplicar a CSPs:

- Assumir uma atribuição com restrições
- não satisfeitas
- Operadores reatribuir valores
- de variáveis

Sem franja! Viva na borda.

▪ Algoritmo: Embora não resolvido,

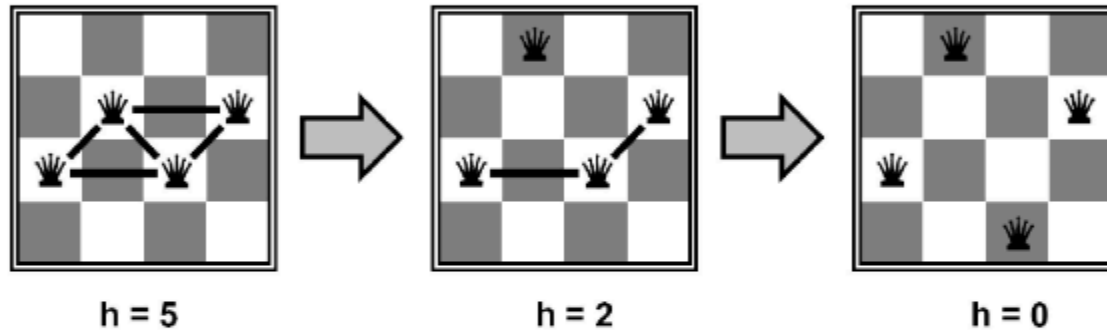
- Seleção de variáveis: selecione aleatoriamente qualquer variável
- conflituosa

Seleção de valor: heurística de conflitos mínimos:

- escolha um valor que viole as menores restrições

▪, ou seja, subida de colina com $h(n)$ = número total de restrições violadas

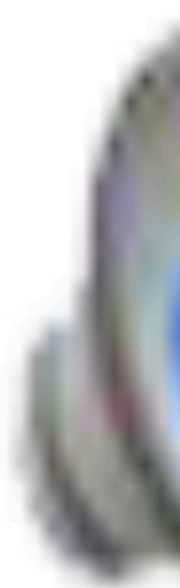
Exemplo: 4-Damas



- Estados: 4 damas em 4 colunas ($4! = 24$ estados)
- Operadores: mova a dama na coluna
- Teste de gol: sem ataques
- Avaliação: $c(n)$ = número de ataques

Vídeo de Melhoria Iterativa de Demonstração – n

Queens



Vídeo de Melhoria Iterativa de Demonstração – Coloração

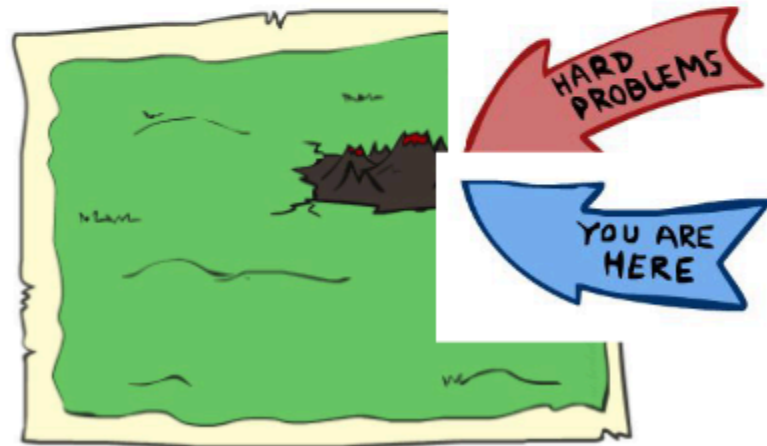
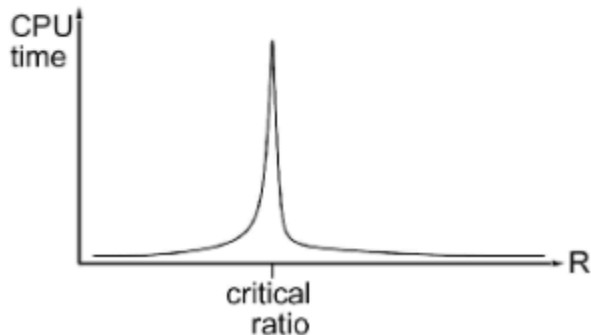


Desempenho dos Min-Conflitos

- Dado o estado inicial aleatório, pode resolver n-rainhas em tempo quase constante para arbitrário n com alta probabilidade (por exemplo, n = 10.000.000)!
- O mesmo parece ser verdade para qualquer CSP gerado aleatoriamente, exceto em um caso restrito

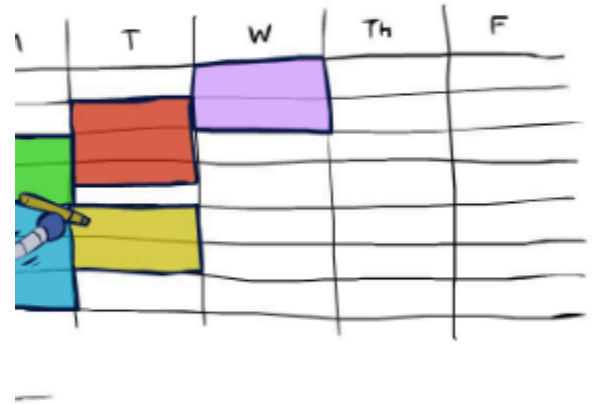
Intervalo da razão

$$R = \frac{\text{number of constraints}}{\text{number of variables}}$$



Resumo: CSPs

- CSPs são um tipo especial de problema de busca:
- Estados são atribuições parciais
- Teste de objetivo definido por restrições
- Solução básica: busca de retrocesso
- Aceleração:
- Ordenação
- Estrutura de filtragem
- Conflitos mínimos iterativos são frequentemente eficazes na prática



Busca Local



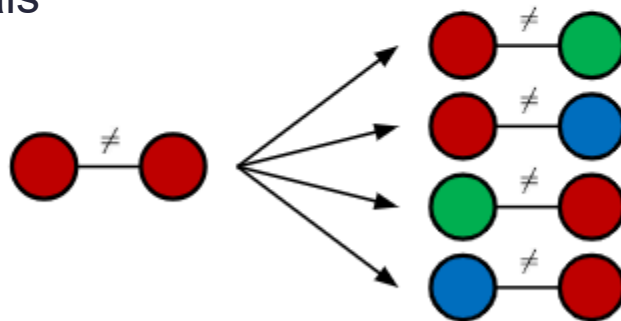
Busca Local



Foto: Justin Merle

Busca Local

- A busca por árvores mantém alternativas inexploradas na periferia (garante completude)
- Busca local: melhore uma única opção até não conseguir melhorá-la (sem franja!).
- Nova função sucessora: mudanças locais



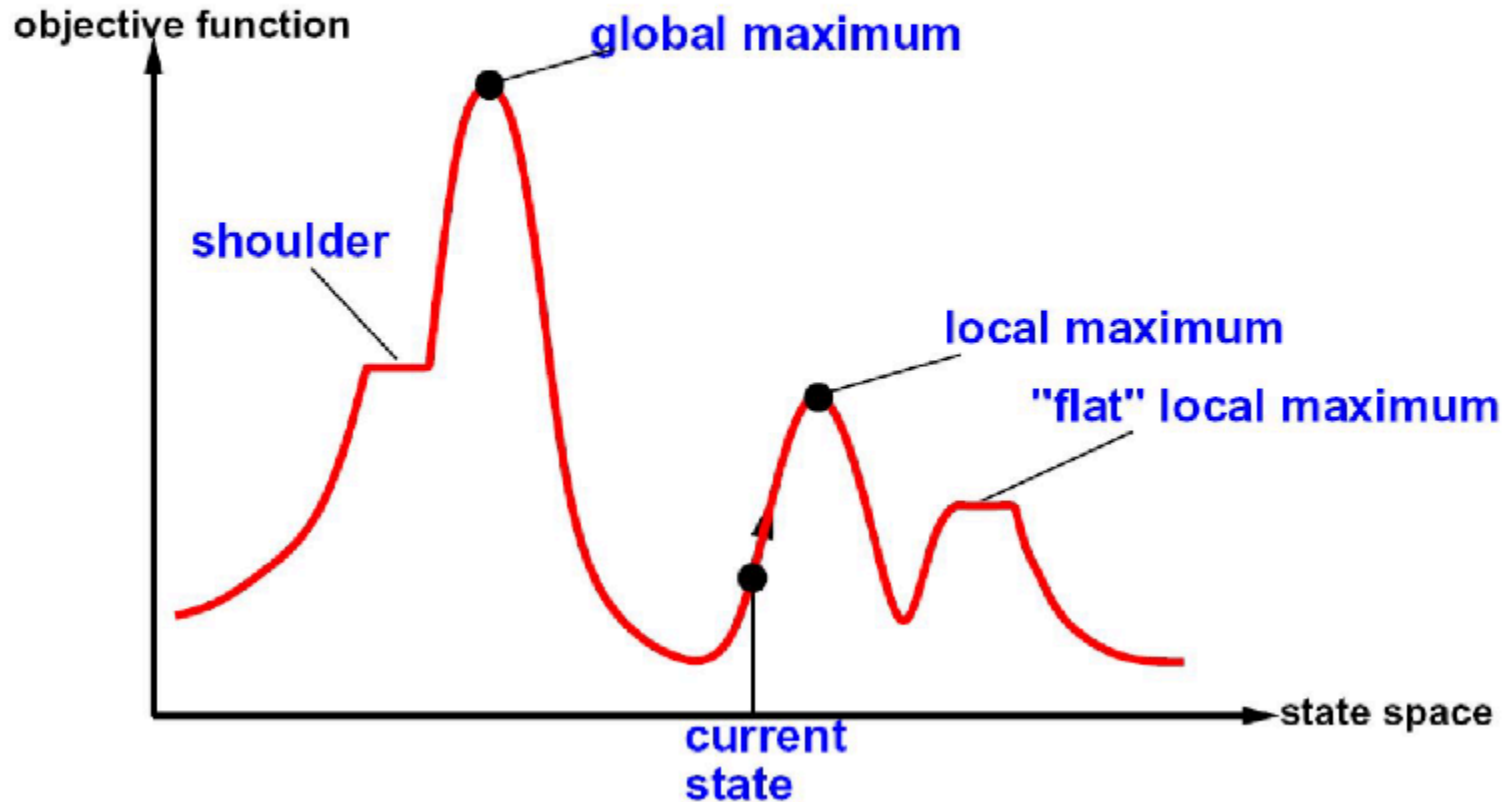
- Geralmente muito mais rápido e eficiente em memória (mas incompleto e subótimo).
- Muitos algoritmos de aprendizado de máquina usam algum tipo de busca local

Escalada de Colinas

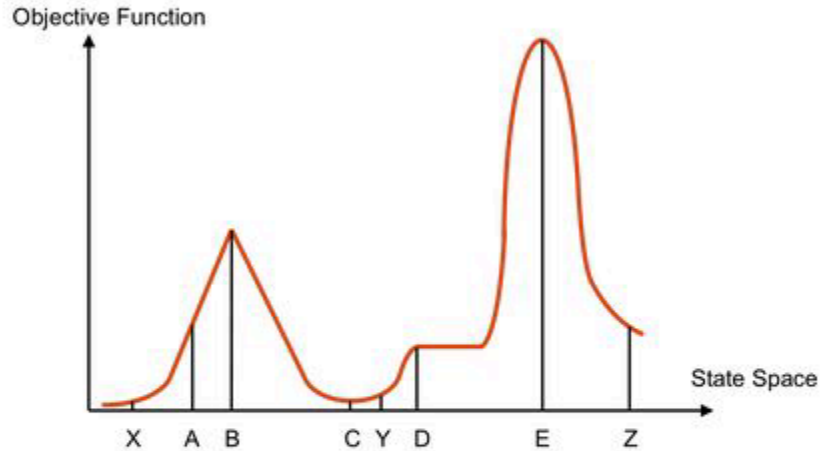
▪ Ideia simples e geral: ▪ começa onde quiser▪. Repita: vá para o melhor estado▪ vizinho. Se não houver vizinhos melhores que o atual, pare▪ o que há de ruim nessa abordagem? ▪ Completa? ▪ Ideal? ▪ O que há de bom nisso? ▪ Fácil de aplicar?



Diagrama de Subida de Colinas



Quiz de Escalada de Colinas



Começando por X, onde você acaba?

Começando pelo Y, onde você acaba?

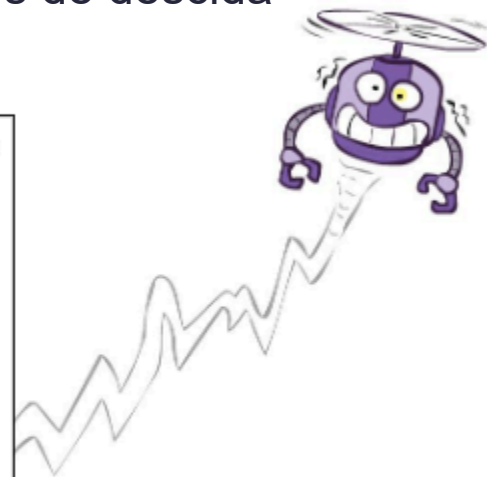
Começando pelo Z, onde você acaba?

Recozimento simulado

- Ideia: Escapar dos máximos locais permitindo movimentos de descida
 - Mas torne-os mais raros com o passar do tempo

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to "temperature"
  local variables: current, a node
                   next, a node
                   T, a "temperature" controlling prob. of downward steps

  current ← MAKE-NODE(INITIAL-STATE[problem])
  for t ← 1 to ∞ do
    T ← schedule[t]
    if T = 0 then return current
    next ← a randomly selected successor of current
     $\Delta E \leftarrow \text{VALUE}[\textit{next}] - \text{VALUE}[\textit{current}]$ 
    if  $\Delta E > 0$  then current ← next
    else current ← next only with probability  $e^{\Delta E / T}$ 
```



Recozimento simulado

- Garantia teórica: ▪

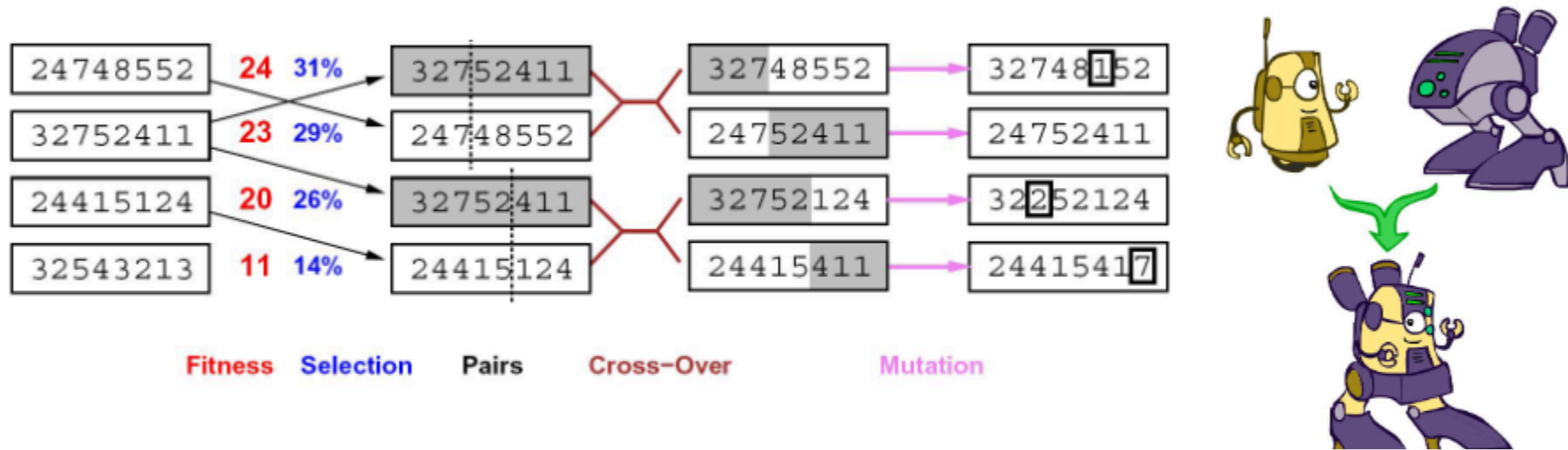
Distribuição estacionária: $p(x) \propto e^{-\frac{E(x)}{kT}}$

- Se a temperatura diminuir lentamente,
Convergirá para o estado ótimo!

- Isso é uma garantia interessante? ▪ Parece mágica, mas a realidade é a realidade: ▪ quanto mais degraus descendo você precisar para escapar de um ótimo local, menos provável é que você consiga fazer todos em uma linha ▪ de tempo. As pessoas pensam muito sobre operadores de crista, que permitem pular pelo espaço de maneiras melhores

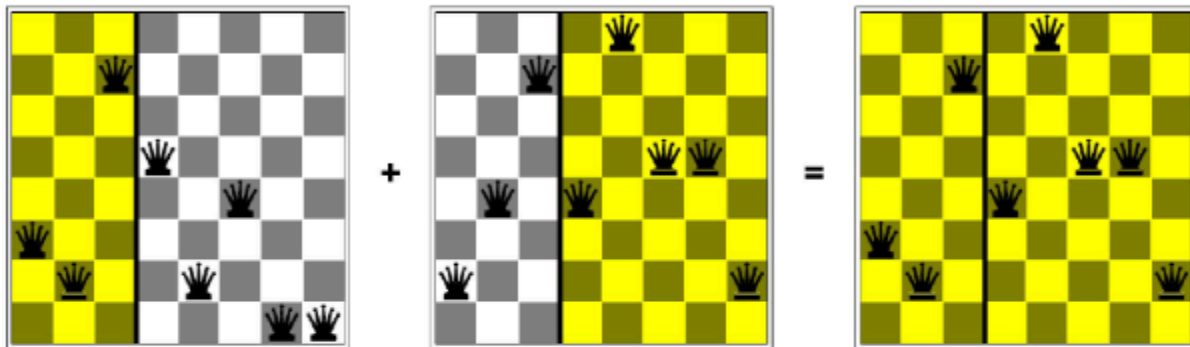


Algoritmos Genéticos



- Algoritmos genéticos usam uma metáfora de seleção natural. Mantêm as melhores N hipóteses em cada etapa (seleção) com base em uma função de aptidão também possuem operadores de cruzamento em pares, com mutação opcional para dar variedade. Possivelmente a técnica mais mal compreendida, mal aplicada (e até menosprezada) do mundo

Exemplo: N-Damas



- Por que o crossover faz sentido aqui? ▪ Quando não faria sentido? ▪ O que seria mutação? ▪ Qual seria uma boa função de aptidão física?

Próxima vez: Busca Adversarial!
